Cookie Clicker + Chess: Clickmate

Summary:
The project will be developing a web based game that combines incremental clicking mechanics like cookie clicker with the gameplay of chess. Players will accumulate resources through clicking during a timed phase, then use said resources to unlock and place chess pieces before playing a match.

Goals and Non-goals:

- Goals:
    - Clicking phase
        - As a player, i want to click repeatedly within a time limit so that i can generate pawn currency
        - Given the game has started, when the timer is active, then each click increases the pawn count
        - Given upgrades are unlocked, when i click, then i may generate more pawns per click
    - Unlock phase
        - As a player, i want to spend pawns to unlock chess pieces or power ups
        - Given sufficient pawns, when i purchase a piece, then it becomes available for placement
        - Given sufficient pawns, i have the option of buy certain powerups which augment my pieces or the gameplay
    - Placement phase
        - As a player, i want to place my unlocked pieces strategically on the board before the match begins
        - Given unlocked pieces, when i drag them onto the board, then their positions are saved
    - Chess phase
        - As a player, i want to play a standard chess game using my customized setup
        - Given both players are ready, when the game starts, then normal chess rules apply
    - Data retention:
        - As a player, I want to be able to keep track of my game history and win loss ratio in the game
        - Given the player has logged in using their credentials, the game must be able to keep track of their match history as well as their wins and losses
- Quality attribute requirements
    - Performance: clicking phase must be able to handle high frequency input without lag. The game should be able to update boardstate without buffering or lag. User input should be registered and updated quickly
    - Fairness: game state must be synchronized and validated server-side
    - Scalability: support for multiple concurrent matches

    - Usability: simple ui for switching between states
- Non-goals
    - Full chess engine innovation (use existing libraries)
    - Complex animations or 3d rendering
    - Large scale multiplayer support

High Level Design:

- System Architecture:
    - The game will use a client-server architecture. The frontend will run in the browser and handle the user interface, clicking interactions, piece selection, drag-and-drop placement, and chessboard display. The backend will manage player authentication, matchmaking, game state validation, timers, economy calculations, and match history. A database will store user accounts, win/loss records, match history, and possibly saved statistics such as average pawns generated or most-used pieces.
    - The frontend should not be trusted to determine the final game result or validate important actions. Instead, the frontend sends player actions to the backend, and the backend verifies whether those actions are legal. This is especially important during the clicking phase and chess phase, where cheating or desynchronization could affect fairness.
- Core Game Flow: The game follows five major phases:
    - Match Start: Two players are matched together or enter a custom lobby. Once both players are ready, the game initializes a shared match state.
    - Clicking Phase: Players click as many times as possible within a fixed time limit to generate pawn currency. The client displays the current pawn count, timer, and any active upgrades. The backend validates the total generated currency to prevent impossible click rates.
    - Unlock Phase: Players spend pawn currency to purchase chess pieces and powerups. Purchased pieces are added to the player's available inventory. Powerups may affect the later placement phase or chess phase.
    - Placement Phase: Players drag unlocked pieces onto their side of the board. Placement rules restrict where pieces can be placed, such as only on the player's first few ranks. The backend stores and validates the final board setup.
    - Chess Phase: The game transitions into a chess match using the customized board setups. Standard chess rules apply unless modified by purchased powerups. Each move is validated using a chess rules library on the backend.
    - End Game: The game ends by checkmate, resignation, timeout, or other defined victory conditions. The result is saved to the database and the player's win/loss record is updated.
- Core Components:
    - Frontend Components:
        - Game Lobby UI: allows players to start matchmaking or join a custom match.
        - Clicking UI: displays timer, pawn count, click button, and active upgrades.

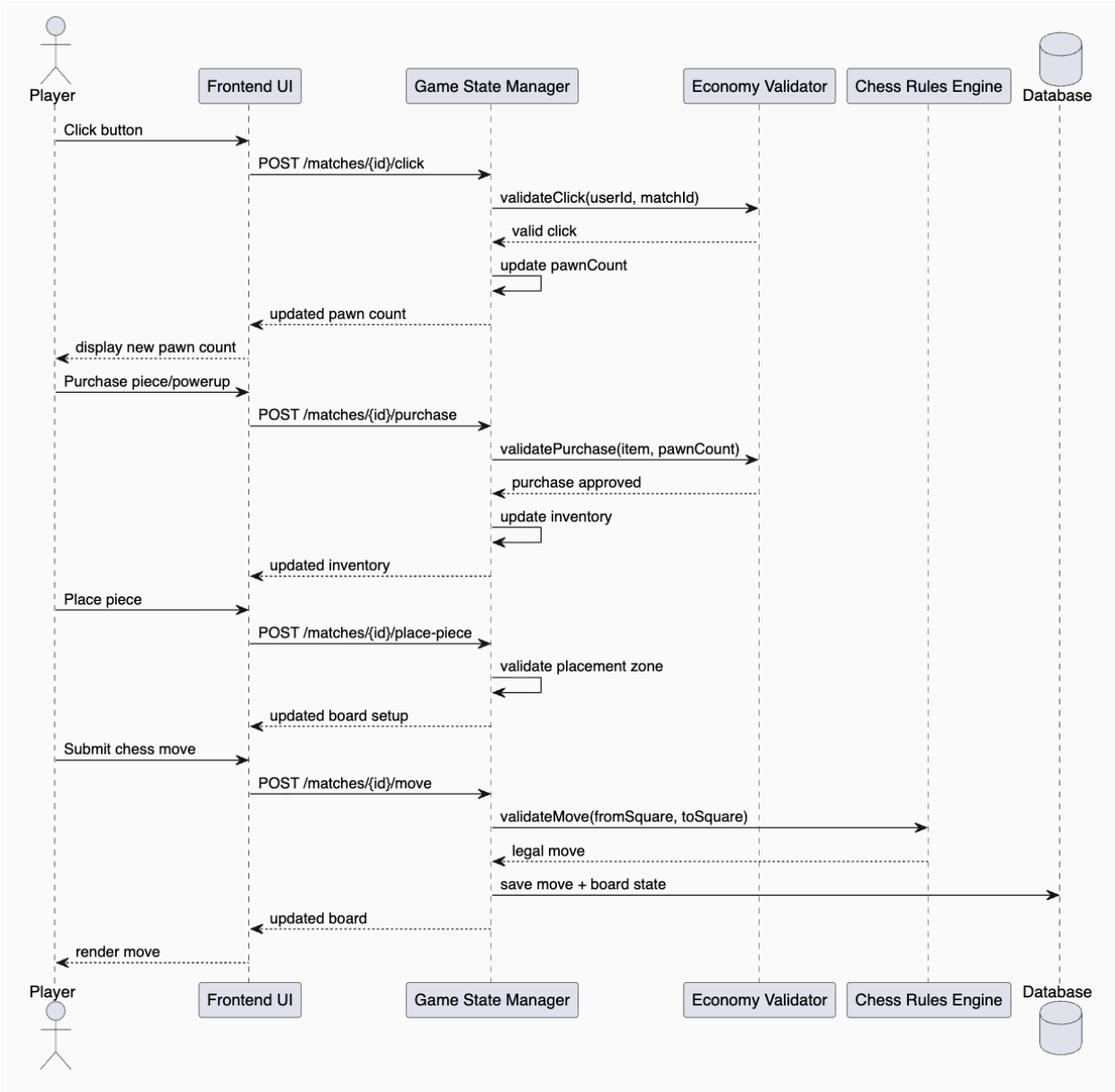
        - Shop UI: allows players to buy pieces and powerups.
        - Placement UI: provides drag-and-drop interaction for placing unlocked pieces.
        - Chessboard UI: displays the active chess game and legal moves.
        - Match Summary UI: shows winner, final board state, match stats, and updated record.
    - Backend Components:
        - Authentication Service: manages login, user identity, and player sessions.
        - Matchmaking Service: pairs players for matches.
        - Game State Manager: tracks the current phase, timers, player resources, board state, and match status.
        - Economy Validator: validates pawn generation, purchases, and spending.
        - Chess Rules Engine: validates legal chess moves and detects check, checkmate, stalemate, and illegal positions.
        - Anti-Cheat Service: detects impossible click rates, suspicious timing patterns, or invalid client actions.
        - Match History Service: records completed games, win/loss outcomes, and player statistics.
- Data Model: The system should include the following major entities:
- User:
    - userId
    - Username
    - passwordHash
    - Wins
    - Losses
    - createdAt
- Match:
    - matchId
    - playerOneId
    - playerTwoId
    - currentPhase
    - startTime
    - winnerId
    - status
- PlayerMatchState:
    - matchId
    - userId
    - pawnCount
    - purchasedPieces
    - purchasedPowerups
    - placedPieces
    - remainingTime
- Piece:
    - pieceType

    - Cost
    - Quantity
    - position
- Powerup:
    - powerupType
    - Cost
    - Effect
    - duration or condition
- Move:
    - matchId
    - userId
    - fromSquare
    - toSquare
    - Timestamp
    - resultingBoardState
- API Design:
- POST /auth/register
    - Creates a new user account.
- POST /auth/login
    - Logs in an existing user.
- POST /matchmaking/join
    - Adds a player to the matchmaking queue.
- POST /matches/{matchId}/click
    - Registers a click during the clicking phase.
- POST /matches/{matchId}/purchase
    - Purchases a piece or powerup using pawn currency.
- POST /matches/{matchId}/place-piece
    - Place an unlocked piece onto the board.
- POST /matches/{matchId}/ready
    - Marks the player as ready for the next phase.
- POST /matches/{matchId}/move
    - Submits a chess move during the chess phase.
- GET /matches/{matchId}
    - Returns the current match state.
- GET /users/{userId}/history
    - Returns the player's match history and win/loss record.
- State Synchronization:
    - The backend is the source of truth for all important game state. The frontend may show immediate updates for responsiveness, but the backend must confirm whether actions are valid. WebSockets can be used for real-time updates, including timer changes, opponent actions, board updates, and phase transitions.
- Game Balance:
    - To avoid making the chess phase irrelevant, the economy should include limits and balancing constraints. For example, each player may have a maximum

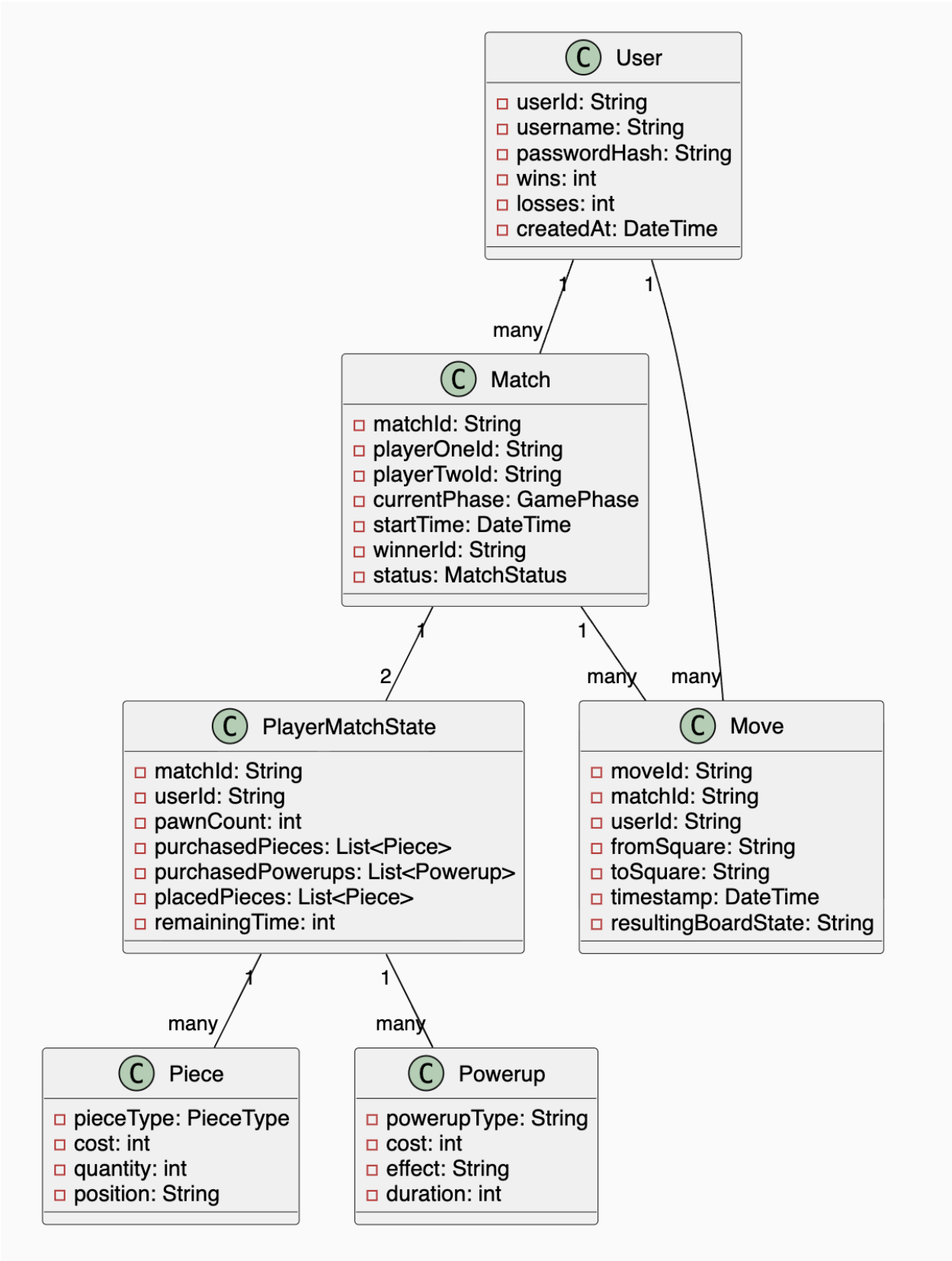
number of queens, repeated purchases may become more expensive, and powerups may be limited to one or two per match. The game should reward clicking and strategy, but not allow the clicking phase to completely determine the winner.

- Security and Fairness:
    - The backend should reject impossible actions, such as buying pieces without enough pawns, placing pieces outside legal zones, making illegal chess moves, or generating an unrealistic number of clicks. Anti-cheat checks should compare click rate, timing patterns, and total pawn generation against reasonable human limits.
- Scalability:
    - The game should support multiple simultaneous matches by storing each match as an independent game session. Since each match has a small number of players, the system can scale by running multiple backend instances and storing active match state in memory or a fast shared store such as Redis. Completed match records can be stored in a persistent database.
- Usability:
    - The user interface should make each phase clear. Players should always know the current phase, remaining time, available resources, and next required action. The transition between clicking, purchasing, placement, and chess should be simple so that players do not get confused by the hybrid game structure.

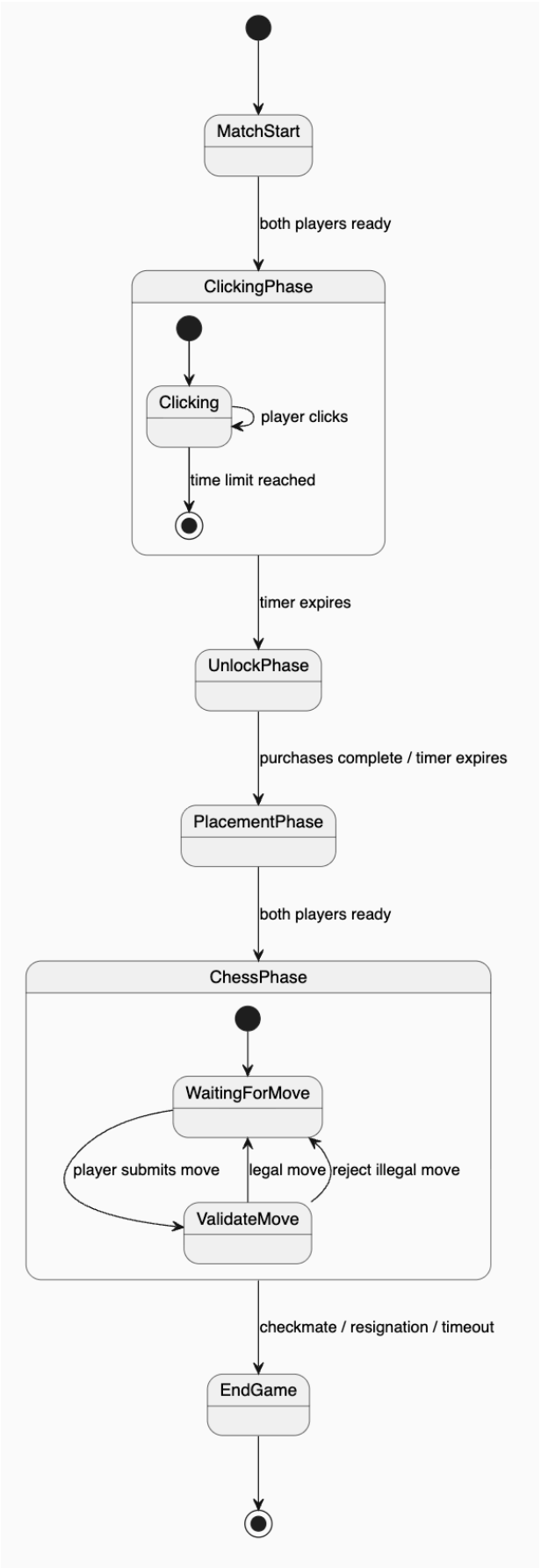
Sequence diagram:



Class diagram:



State machine diagram:


MatchStart
both players ready
ClickingPhase
Clicking
player clicks
time limit reached
timer expires
UnlockPhase
purchases complete / timer expires
PlacementPhase
both players ready
ChessPhase
WaitingForMove
player submits move
legal move
reject illegal move
ValidateMove
checkmate / resignation / timeout
EndGame

Frontend
- React (UI + state management)
- Tailwind CSS (styling)
- Socket.IO client (real-time updates)

Backend
- Node.js + Express (API server)
- Socket.IO (real-time communication)
- Chess.js (move validation)
- Redis (active match state + timers)

Database
- PostgreSQL (persistent data: users, matches)
- Redis (ephemeral state: active games)

Infrastructure
- Docker (containerization)
- AWS / GCP (deployment)
- Nginx (reverse proxy)

Detailed Design Decisions
- Currency system
    - Base unit: pawn
    - Conversion:
        - Pawn -> pieces (knight, bishop, etc.)
        - Pawn -> powerups
- Piece costs
    - Knight: 25 pawns
    - Bishop: 25 pawns
    - Rook: 50 pawns
    - Queen 150 pawns
- Powerups
    - More time
    - Opponent has less time
    - Each move give back more time
    - Pawns move in more directions
    - Limit opponent board space

Alternatives considered:
- Standard Chess
    - An obvious alternative to our modified chess is standard chess without the economy layer. This is an obvious alternative as it has a much simpler implementation and has easier onboarding, but this is not much of a valid alternative because it completely removes the novelty of our project and removes

the progression mechanic. This is also not much different from existing chess platforms like chess.com or lichess.

- Idle Game without the Chess Phase
    - Kind of like cookie clicker, this is a purely incremental game that has players just clicking to generate resources and unlock upgrades. This is much simpler to implement and has a clear progression loop, but lacks any complexity and doesn't have the multiplayer element that we are looking for. Also, this has been implemented in many cases thus is repetitive.
- Replacing turn based chess with RTS board
    - Instead of having turn by turn chess, we can have it where people play as fast as they can move/think. This makes it much more dynamic and fast paced, and has the possibility of adding strong synergy with certain resources generation mechanics, but is hard to implement and test.
- Predefined loadout instead of free placement
    - People get to select presets rather than place pieces however they want. This removes the possibility of OP setups and a bland meta for top players, as we can rotate out loadouts and make them as fair as possible (hard counters, etc). However, this limits creativity and uniqueness, and given how creative chess is already, we want to keep this creativity.

Risks and mitigations:

- Unbalanced economy/Game Balance
    - Clicking is what can heavily contribute to the outcome, essentially making the chess phase irrelevant. Players who generate more currency will just naturally always win. A mitigation we can have is imposing maxes/caps on the number of type of pieces (ex. Limited queens), having increasing costs for repeated upgrades/piece costs (ex. 100→200→400…), etc.
- Cheating
    - Autoclickers give players an inherent advantage as games like cookie clicker already have many players abusing this method. While something like this is more accepted in a single player environment, our game is multiplayer, hence players that have faster click speeds (inhuman) gain unfair advantages. By adding soft caps, passive income generators, normalization, and auto cheat detection systems, we can help limit it while keeping clicking competitive.
- Player Fatigue/Repetitiveness
    - Repetitive clicking is boring and fatiguing, leading to burnout. By adding idle mechanics, random events, and alternative progression paths and constant updates, we can help mitigate this.
- Matchmaking Fairness
    - Adding MMR based matchmaking and separating quickplay from competitive play will keep players playing and keep the game fresh.

Future Extensions:

- Ranked ladder system

- AI opponents
- Custom game modes
- Additional powerups

Feasibility:
The proposed application is feasible to implement within the project timeline because it leverages well-established technologies and existing libraries to reduce development complexity. For example, instead of building a chess engine from scratch, we can use mature libraries like chess.js for move validation, allowing us to focus on the novel game mechanics. While some APIs and tools (e.g., WebSockets for real-time updates) may be initially unfamiliar, they are widely documented and commonly used in web applications, making them manageable to learn. To avoid scope creep, we will prioritize a minimum viable product (MVP) that includes the core phases (clicking, purchasing, placement, and chess) and defer advanced features like ranked matchmaking or complex powerups. Costs will remain minimal since the project can be developed and tested locally with free-tier cloud services if needed. Although similar games (e.g., chess platforms or idle clickers) already exist, our hybrid design differentiates itself through the integration of resource generation and strategic gameplay. Finally, performance requirements are modest and achievable, as the system only needs to handle small-scale matches with limited players, making it realistic to deliver a functional and responsive demo within the given timeframe.